



MENTEMOVIMENTO
ASSOCIAÇÃO
PRÓ-SAÚDE MENTAL

MANUAL DO PROGRAMADOR

Manutenção técnica da aplicação

Onde está o projeto, como funciona a arquitetura, como atualizar o website e que cuidados manter para a associação não depender de uma única pessoa.

CÓDIGO

GitHub: [jf77ps/Socios_MenteMovimento](#)

PUBLICAÇÃO

Vercel, branch main

BASE DE DADOS

Supabase PostgreSQL

O que este manual explica

1. Arquitetura geral

2. Onde estão os ficheiros

3. Como testar localmente

4. Como publicar alterações

5. Supabase e SQL

6. Funções seguras da Vercel

7. Segurança, passwords e backups

8. Plano semestral de cibersegurança

9. Alterações futuras

10. Transferência de responsabilidade

11. Checklist técnica

Este manual é para a pessoa que fica responsável por manter o site, corrigir pequenos problemas e coordenar futuras alterações técnicas.

1. ARQUITETURA

Como o projeto está montado

Frontend

Website estático com `index.html`, `styles.css` e `app.js`.

Base de dados

Supabase PostgreSQL guarda sócios, utilizadores autorizados e histórico.

Autenticação

Supabase Auth trata login por email e password.

Publicação

Vercel publica automaticamente quando há push para o GitHub.

Funções API

A pasta `api/` contém operações que precisam de chave privada no servidor.

Segurança

Row Level Security no Supabase, headers no `vercel.json` e perfis de acesso na app.

2. FICHEIROS

Onde está o código

Pasta local: `C:\Users\Asus\Documents\New project`

GitHub: `https://github.com/jf77ps/Socios\_MenteMovimento`

Website: `https://socios-mente-movimento.vercel.app`

FICHEIRO OU PASTA

FUNÇÃO

`index.html`

Estrutura da aplicação: login, painel, modais e formulários.

`styles.css`

Aspeto visual da aplicação, temas claro/escuro e responsividade.

`app.js`

Lógica principal: Supabase, permissões, sócios, quotas, exportação e idiomas.

`config.js`

URL e chave pública do Supabase. Nunca colocar aqui a service role key.

`api/`

Funções de servidor para criar e eliminar utilizadores.

`supabase/`

Scripts SQL para criar ou alterar tabelas, políticas e funções.

`docs/`

CSS dos manuais, imagens e PDFs de documentação.

`vendor/`

Bibliotecas externas guardadas localmente, evitando dependência de CDN.

`vercel.json`

Configuração da Vercel, headers de segurança e regras das APIs.

3. TESTES LOCAIS

Como testar antes de publicar

1. Abrir a pasta do projeto no editor.
2. Iniciar um servidor local na pasta do projeto, por exemplo `python -m http.server 4177`.
3. Abrir `http://127.0.0.1:4177` no browser.
4. Testar login, pesquisa, criar/editar sócio, pagar quota, exportar e gestor de administradores.
5. Validar o JavaScript com `node --check app.js`.
6. Confirmar que não existem erros visíveis na consola do browser.

Nunca testar alterações diretamente em produção sem antes confirmar localmente que o fluxo principal continua a funcionar.

4. PUBLICAÇÃO

Como atualizar o site

1. Alterar os ficheiros necessários.
2. Testar localmente.
3. Fazer commit no Git com uma mensagem clara.
4. Enviar para o GitHub no branch `main`.
5. A Vercel faz o deploy automaticamente.
6. Abrir o website publicado e confirmar a alteração.

Fluxo esperado: `computador local` → `GitHub` → `Vercel` → `website público`.

5. SUPABASE

Base de dados, SQL e RLS

O código do site não altera sozinho a estrutura da base de dados. Quando há um campo novo, também é preciso executar SQL no Supabase.

1. Entrar no Supabase.
2. Abrir o projeto da associação.
3. Ir a **SQL Editor**.
4. Colar o script da pasta `supabase/`.
5. Clicar em **Run**.
6. Testar no site publicado.

SCRIPT	QUANDO USAR
<code>schema.sql</code>	Criação inicial da base de dados.
<code>harden-rls.sql</code>	Reforço das permissões e Row Level Security.
<code>add-quota-paid-at.sql</code>	Adiciona a data em que a quota foi paga.
<code>add-approval-minute-number.sql</code>	Adiciona o N° de Ata de Aprovação.

6. APIS

Funções seguras da Vercel

Algumas ações não devem ser feitas diretamente no browser porque precisam de permissões fortes. Por isso existem funções na pasta `api/`.

`api/create-user.js`

Cria novos utilizadores autorizados através do Gestor de Administradores.

`api/delete-user.js`

Remove utilizadores com validações para impedir ações inseguras.

A variável `SUPABASE_SERVICE_ROLE_KEY` fica apenas na Vercel, em Environment Variables. Nunca deve ir para GitHub, `config.js` ou qualquer ficheiro público.

7. SEGURANÇA

Cuidados obrigatórios de cibersegurança

- ✓ Manter Row Level Security ativa no Supabase.
- ✓ Dar acesso administrativo apenas a quem realmente precisa.
- ✓ Ativar 2FA/MFA nas contas GitHub, Vercel e Supabase.
- ✓ Guardar passwords num local seguro e controlado pela associação.
- ✓ Fazer backups/exportações regulares dos dados e testar recuperação.
- ✓ Não colocar chaves privadas no código público.
- ✓ Rever o histórico de alterações quando houver dúvidas.
- ✓ Remover acessos quando uma pessoa sai da associação.

Regra prática: cada pessoa deve ter a sua própria conta. Evitar contas partilhadas; quando forem inevitáveis, guardar a password num gestor seguro e mudar a password sempre que alguém com acesso sair da associação.

8. ROTINA SEMESTRAL

Plano semestral de cibersegurança

De 6 em 6 meses deve existir uma revisão técnica formal, registada por escrito com data, responsável e resultado.

TAREFA	O QUE FAZER	REGISTO MÍNIMO
Backups	Exportar dados dos sócios e confirmar que existe cópia protegida fora da conta principal.	Data, responsável, local da cópia e ficheiro exportado.
Teste de recuperação	Confirmar que o ficheiro exportado abre corretamente e que os dados essenciais estão presentes.	Resultado do teste e erros encontrados.
Passwords administrativas	Rodar passwords das contas institucionais/admin e chaves partilhadas, evitando variações previsíveis.	Contas revistas e confirmação de atualização.
Acessos	Rever GitHub, Vercel, Supabase e Gestor de Administradores da app.	Lista de pessoas que ficaram com acesso.
MFA/2FA	Confirmar que administradores têm autenticação de dois fatores ativa.	Serviços verificados e contas sem 2FA.
Histórico e logs	Rever alterações recentes, eliminações e ações administrativas suspeitas.	Resumo da revisão e ações tomadas.

Além da revisão semestral, mudar passwords imediatamente se houver suspeita de acesso indevido, perda de computador/telemóvel, saída de alguém com acesso, ou partilha accidental de credenciais.

8.1 BACKUPS

Política de backups

Periodicidade mínima

Backup semestral obrigatório. Se houver muitas alterações de sócios/quotas, fazer também backup mensal.

O que guardar

Exportação da lista de sócios, registo de quotas, utilizadores autorizados e scripts SQL importantes.

Onde guardar

Duas cópias: uma em armazenamento institucional protegido e outra em local separado, com acesso restrito.

Proteção

Evitar enviar backups por email comum. Se possível, guardar em pasta protegida e com controlo de acessos.

Um backup só é útil se puder ser recuperado. Pelo menos uma vez por semestre, abrir o ficheiro exportado e confirmar que contém nomes, contactos, quotas e campos principais.

8.2 PASSWORDS

Política de passwords e contas

- ✓ Usar passwords únicas para GitHub, Vercel, Supabase, email institucional e app.
- ✓ Usar frases-password longas ou passwords geradas por gestor de passwords.
- ✓ Ativar 2FA/MFA em todas as contas administrativas.
- ✓ Fazer revisão semestral das passwords administrativas e institucionais.
- ✓ Mudar passwords partilhadas sempre que alguém com acesso deixa a associação.
- ✓ Não reutilizar passwords antigas nem criar variações óbvias, como mudar apenas o ano ou um número.
- ✓ Não guardar passwords em documentos públicos, chats, folhas de cálculo abertas ou emails.
- ✓ Desativar contas que já não são usadas.

Nota técnica: recomendações modernas de cibersegurança dão prioridade a passwords únicas, longas, MFA e mudança imediata quando há suspeita de compromisso. A revisão semestral deve evitar criar passwords fracas ou previsíveis.

8.3 INCIDENTES

O que fazer se houver suspeita de problema

1. Suspender ou remover o acesso da conta suspeita no Gestor de Administradores.
2. Mudar passwords das contas afetadas e confirmar 2FA/MFA.
3. Rodar a `SUPABASE_SERVICE_ROLE_KEY` se houver suspeita de exposição da chave.
4. Rever histórico da app, utilizadores autorizados e logs do Supabase/Vercel.
5. Fazer backup do estado atual antes de alterações profundas.
6. Confirmar se houve acesso a dados pessoais dos sócios.
7. Informar a direção e avaliar obrigações RGPD com apoio adequado.
8. Registrar o incidente: data, pessoa que detetou, impacto, ações tomadas e conclusão.

Nunca apagar logs ou histórico para "limpar" o problema. Primeiro preservar evidências, depois corrigir o acesso.

8.4 REFERÊNCIAS

Referências úteis de cibersegurança

FONTE	UTILIDADE PARA ESTE PROJETO
CISA - Back Up Business Data	Planeamento de backups e recuperação após incidentes.
CISA - Stop Ransomware Guide	Boas práticas para reduzir impacto de ransomware e testar backups.
NCSC - Password policy	Orientação moderna para passwords, evitando regras que levem a passwords fracas.
NIST Digital Identity Guidelines	Referência técnica para autenticação, passwords e gestão de identidade.

Estas referências não substituem apoio jurídico ou técnico especializado, mas ajudam a manter o projeto alinhado com boas práticas atuais.

9. ALTERAÇÕES FUTURAS

Como adicionar um campo novo

1. Criar o SQL na pasta `supabase/`.
2. Executar o SQL no Supabase.
3. Adicionar o campo ao formulário em `index.html`.
4. Atualizar leitura, gravação, validação e exportação em `app.js`.
5. Atualizar textos de português/inglês se o campo aparecer no interface.
6. Atualizar `styles.css` se o layout precisar.
7. Atualizar manuais e relatórios.
8. Testar localmente e publicar pelo GitHub.

10. CONTINUIDADE

Como a associação deixa de depender de uma pessoa

GitHub

Transferir o repositório para uma organização GitHub da associação.

Vercel

Transferir o projeto para uma equipa Vercel controlada pela associação.

Supabase

Garantir que a organização/projeto Supabase pertence à associação.

Acessos

Ter pelo menos duas pessoas com acesso administrativo e 2FA ativo.

O ideal é usar emails institucionais da associação para propriedade, faturação, recuperação de conta e alertas técnicos.

11. CHECKLIST TÉCNICA

Antes de publicar qualquer alteração

- ✓ O pedido está claro e limitado ao necessário?
- ✓ Os ficheiros alterados são os corretos?
- ✓ O site foi testado localmente?
- ✓ `node --check app.js` passou sem erros?
- ✓ Se houve campo novo, o SQL foi criado e executado?
- ✓ As permissões continuam corretas para admin, operador e consulta?
- ✓ O commit foi enviado para o GitHub?
- ✓ A Vercel publicou e o site final foi confirmado?

Versão do manual: 22/05/2026.